

C++中 const 修饰指针的用法及场景解析

我来给你把这三种 `const` 修饰指针的情况，从语法到使用场景，讲得明明白白。

一、核心规则：`const` 离谁近，就修饰谁

判断 `const` 修饰的是「指针本身」还是「指针指向的内容」，记住一个简单原则：

- `const` 写在 `*` 的**左边** → 修饰的是「指针指向的内容」（内容不可改）
- `const` 写在 `*` 的**右边** → 修饰的是「指针本身」（指针不可改）

下面我们逐个拆解你代码里的三种情况：

1. `const Student* p` （最常用）

- 写法：`const` 在 `*` 左边，修饰 `Student`
- 含义：
 - 指针 `p` 本身的值（地址）**可以修改**
 - 指针 `p` 指向的 `Student` 结构体的内容**不可以修改**
- 举个例子：

代码块

```
1  Student s1 = {18, "张三"};
2  Student s2 = {19, "李四"};
3  const Student* p = &s1;
4
5  p = &s2;    //  合法：指针p本身可以指向别的Student
6  p->age = 20; //  非法：不能通过p修改它指向的结构体内容
```

- 使用场景：函数只需要读取结构体内容，不需要修改，用它来做“只读参数”，是函数传参里最推荐的写法。

2. `Student* const p`

- 写法：`const` 在 `*` 右边，修饰指针 `p`

- 含义：

- 指针 `p` 本身的值（地址）**不可以修改**（初始化后就固定指向同一个地址）
- 指针 `p` 指向的 `Student` 结构体的内容**可以修改**

- 举个例子：

代码块

```
1 Student s1 = {18, "张三"};
2 Student s2 = {19, "李四"};
3 Student* const p = &s1;
4
5 p = &s2; // ❌ 非法：指针p本身不能修改指向
6 p->age = 20; // ✅ 合法：可以通过p修改它指向的结构体内容
```

- 使用场景：指针本身需要固定指向一个对象，但是允许修改对象的内容。这种场景在实际开发中比第一种少见很多。

3. `const Student* const p`

- 写法：`const` 同时在 `*` 的左边和右边

- 含义：

- 指针 `p` 本身的值（地址）**不可以修改**
- 指针 `p` 指向的 `Student` 结构体的内容**也不可以修改**

- 举个例子：

代码块

```
1 Student s1 = {18, "张三"};
2 Student s2 = {19, "李四"};
3 const Student* const p = &s1;
4
5 p = &s2; // ❌ 非法：指针p本身不能修改指向
6 p->age = 20; // ❌ 非法：也不能修改结构体内容
```

- 使用场景：函数既不允许修改指针指向，也不允许修改指针指向的内容，是“完全只读”的参数，通常在底层 API 或严格的只读接口中使用。

二、一张表帮你分清三种情况

函数声明	指针本身能否修改？	指向的结构体内容能否修改？	核心用途
<code>const Student* p</code>	✅ 可以	❌ 不可以	只读访问结构体（最常用）
<code>Student* const p</code>	❌ 不可以	✅ 可以	固定指向、但可修改内容
<code>const Student* const p</code>	❌ 不可以	❌ 不可以	完全只读，连指针都不能改

三、补充：一个容易搞混的写法

`const Student* p` 和 `Student const* p` 是**完全等价**的，都是修饰指向的内容。
所以你看下面这两种写法，效果一模一样：

代码块

```
1 void fun1(const Student* p);
2 void fun1(Student const* p); // 和上面完全等价
```

四、实际开发里怎么选？

1. 90% 以上的场景用第一种 `const Student* p`：
函数传参时，如果只需要读取结构体数据，一律用它。既能防止你不小心修改数据，又能接受 `Student*` 和 `const Student*` 两种类型的实参，兼容性最好。
2. 第二种 `Student* const p` 几乎不用：
指针本身是局部变量，修改它的值不会影响外面的实参，所以限制它的修改意义不大。
3. 第三种 `const Student* const p` 很少用：
只有在非常严格的接口中才会用，大部分时候第一种就足够了。

如果你愿意，我可以给你写一段完整的可运行代码，把这三种情况的合法 / 非法操作都演示一遍，帮你彻底记牢它们的区别。需要吗？

（注：文档部分内容可能由 AI 生成）